TECHNICAL BRIEF · 2026

Mesh.

Structural source code compression for LLM context efficiency.

Edgar Baumann
Co-Founder · WU Wien

Philipp Horn
Co-Founder · WHU

74%	3.9×	$3.90
avg token reduction	context gain per 128k window	vs $15.00/MTok · Claude Opus 4.6

edgar.baumann@try-mesh.com · philipp.horn@try-mesh.com

CONTEXT DEGRADATION

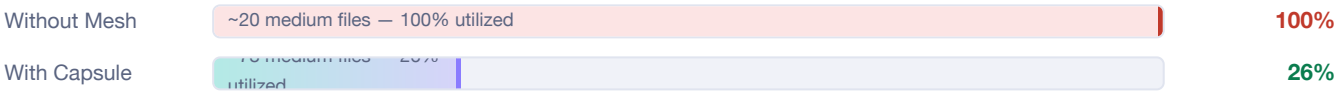
More context doesn't mean better answers — it can mean worse ones.

LLMs operate with a fixed context window. As utilization increases, retrieval accuracy degrades — this is a measured phenomenon, not a hypothesis.

NIAH — Needle In A Haystack

Standard retrieval benchmark: a specific fact is placed at varying positions in a long document; the model must retrieve it. Published research (Liu et al., 2023, "*Lost in the Middle*") shows:

- Models perform reliably below ~60–70% context utilization
- Above that threshold, retrieval accuracy drops **20–40%** depending on where the target appears
- The effect compounds at 100k+ token contexts



SWE-bench

Industry benchmark for AI coding agents solving real GitHub issues. Top models (Claude Opus 4.6, GPT-4o, Gemini 3.1 Pro) resolve approximately **40–55%** of tasks. Root cause analysis of failures consistently identifies one leading pattern: **the agent lacked sufficient codebase context to understand the problem**.

Current tools use embedding-based retrieval: semantic search returns the most relevant files, which are then sent *raw*. This selects what gets loaded, but what is loaded still consumes the full raw token budget. As the codebase grows, per-window coverage shrinks.

ARCHITECTURE

Structural compression with selective recovery.

Capsule processes source files before they reach any LLM. Not summarization — the original file is always reconstructable in full on demand.

Pipeline steps

STEP	WHAT HAPPENS
1 — Parse	File decomposed into structural components: declarations, function signatures, type annotations, doc comments, control flow markers
2 — Compress	Implementation bodies and non-structural content reduced to minimal structural representations
3 — Encode	Compact structural descriptor preserving the semantic skeleton of the file
4 — Selective Recovery	When the model identifies a specific function as relevant, the full implementation is restored on demand

```
User query
    |
    ▼
Workspace indexer     ← scans project, builds file registry
    |
    ▼
Capsule compression   ← structural compression per file
    |
    ▼
Context assembler     ← packs compressed files into context window
    |
    ▼
Model API             ← Claude / Gemini / GPT
    |
    ▼
Selective recovery    ← restores full body on model request
    |
    ▼
Response + diff
```

Capsule activates dynamically. Files below ~200 tokens get a pass-through (format overhead would exceed savings). Production codebases are 95%+ medium-to-large files where compression is highly effective.

edgar.baumann@try-mesh.com · philipp.horn@try-mesh.com

PERFORMANCE

Production pipeline · 5 file types · 6 size tiers

TypeScript, YAML, SQL, HTML, Markdown — real production numbers from the live Capsule pipeline.

FILE SIZE	TOKEN REDUCTION	SAVING
~200B (xs) — pass-through		+10%
~1 KB (small)		–83%
~5 KB (medium)		–95%
~18 KB (large)		–98.5%
~50 KB (XL)		–99.4%
~100 KB (XXL)		–99.9%
Average (small–XXL)		**–74% → 3.9×**

Context window impact (128k)

Without Mesh	~20 medium files — 100% utilized	100%
With Capsule	~78 medium files — 26% utilized	26%

Cost impact (Claude Opus 4.6 — $15.00/MTok input)

SCENARIO	TOKENS SENT	API COST	FILES COVERED
Raw code	1,000,000	$15.00	~100 medium files
With Capsule	**260,000**	**$3.90**	**Same 100 files**

edgar.baumann@try-mesh.com · philipp.horn@try-mesh.com

RETRIEVAL VS COMPRESSION

A fundamental architectural difference.

PROPERTY	EMBEDDING RETRIEVAL (INDUSTRY DEFAULT)	CAPSULE COMPRESSION (MESH)
What reaches the model	Raw file content at full token cost	**Compressed structural descriptor**
Token cost	Scales with file count × file size	**Fixed low cost regardless of file size**
Context window coverage	Limited to what fits raw	**3.9× more files per window**
Implementation bodies	Sent in full	**Compressed, restored on demand**
Codebase size sensitivity	High — coverage degrades as codebase grows	**Low — compression ratio improves with file size**
Approaches mutually exclusive?	No — Capsule is additive on top of any retrieval strategy	

Model compatibility

Capsule output is plain text readable by any transformer LLM. Currently supported:

- **Anthropic** — Claude Opus 4.6, Claude Sonnet (all versions)
- **Google** — Gemini 3.1 Pro, Gemini Flash
- **OpenAI** — GPT-4o, GPT-4o mini, o1, o3

Provider switching is runtime-configurable — no re-indexing or pipeline changes required.

File type support

LANGUAGE	STRUCTURAL ELEMENTS PRESERVED
TypeScript / JS	Exports, class declarations, function signatures, type definitions, interfaces
Python	Module-level declarations, class/function signatures, docstrings
YAML / JSON	Top-level keys, schema structure
SQL	Table/view/function definitions, column names and types
HTML	Document structure, component boundaries
Markdown	Heading hierarchy, code block presence

Go, Rust, Java, C# support planned for Capsule v2.